

Detailed results

0. What is in here, and where each number comes from

source	what it is
exppragvllm	run 1 of the wording experiment
exppragvllm2	run 2, rebuilt (fixes, truncation and errors from exppragvllm)
isolate2	one word swapped in the task line
4 databases from exppragvllm2	what the model wrote, read back

The accuracy claims rest on the second and third. The claims about how the model reasons rest on the fourth, which is much thinner.

NOTE: This document is an in-depth of what was done and how the results were extracted and why the conclusions were drawn, the honest reason for its existence is that the findings were quite a lot to compress into a two-page summary, so this serves as an extension, unlike the summary where words all the text is 100% self-written in the making of this document due to a time crunch Claude's word extension was used.

1. The prompts

1.1 The base document

You are playing 100 rounds against the same opponent.
Each round you both pick one of: J, F, M.
F beats J, M beats F, J beats M. Same pick is a tie.
A win scores +1, a loss -1, a tie 0. Win as many rounds as you can.

Round 1
Opponent move: F
Your move: J
Round 2
Opponent move: M
Your move: F
Round 3
Opponent move: J
Your move: M

You are currently playing round 4.
What do you predict the other player will play in this round?

Opponent move:

The document ends at the bare cue. Whatever token comes next is the measurement. On the arms that read probabilities, nothing is generated at all: the probability mass on J, F and M at that position is read straight off.

Why the opponent's move comes first in each round. Because the cue is the last line, and with the opponent leading, every round in the document opens with Opponent move: followed by a letter. So by the time the model reaches the cue it has seen that exact shape three times. Putting your own move first would leave the cue in a position the history never takes.

Neutral J,F and M labels are used as the model has read a great deal about Rock Paper Scissors, so a good answer might be a memory artifact rather than a deduction. Renaming removes that, and the beats relation is stated once so the model has to use the stated version.

1.2 The line that varies

form	the line, with the verb predict
question	What do you {verb} the other player will play in this round?
statement	{verb} the other player's move in this round.
statement_n	{verb} the other player's move in round 4.
taskverb	Your task is to {verb} the other player's move in this round.
anchor	Which move will the other player play in this round?

Why four forms. Run 1 used eleven questions and one instruction, which meant that if a difference appeared, there was no way to say whether it came from the verb or from the fact that the question was a question. Four forms separate them.

Why `statement\_n` exists. It differs from statement by four words: it names the round. Naming the round is a second manipulation that would otherwise ride along invisibly inside statement. Keeping both makes it one measurable cell per verb instead of a confound.

Why `taskverb` exists. It is run 1's broken condition, opened up. In run 1 there was a single task condition that ran at 0.526 to 0.581 coverage against 0.82 for everything else, and answered J 91.2 percent of the time against tft. One prompt with one verb in it cannot say whether the floor came from the imperative frame or from predict sitting inside it. Now the frame is held and the verb moves.

1.3 The HINT levels

- L0 (nothing)
- L1 Your task is to find the pattern in the history and use it to predict.
- L2 The opponent's move depends on what you played in the previous round.
- L3 L1 + L2

In previous experiments it was deduced that the problem lied because of the models ability to not form the correct hypothesis, and then steering was used with the original task prompt L0, to see if with the help of steering it lands on the correct hypothesis more often, also the L0 – L3 also shows how much steering is needed for the model to end up on the correct hypothesis more reliably.

1.4 The generating arms

In the generation arms one line is added:

Your answer MUST end with a line formatted exactly like:
Opponent move: <one of the moves>

That line constrains the form of the answer and nothing else, so parsing the reasoning for the answer is easy.

1.5 The rule probe

This is also something I carried over from the previous experiments, the point of this was to see, if the model is allowed to carry its reasoning into the N+1 prompt, if it reasons correctly once, does it then have the ability to carry that correct hypothesis to the end. And secondly another phenomena that was tested here was time to belief per level, that with the reasoning enabled how many rounds does it need to come to the correct hypothesis.

This adds one more prompt:

Also state what you currently believe the opponent's strategy is,
on its own line, formatted exactly like:
Rule: <one short sentence>

The sentence is stored verbatim.

2. What was checked before any result was read

2.1 The prompt guards

check	what it asserts	result
only the ask differs	all 38 conditions give one document apart from one line	pass
ask is one line	no phrasing spans two lines	pass
levels nest	L3 is exactly L1 plus L2	pass
nothing to copy	no template or option list beside the cue	pass

These run before any job is submitted, on the assembled document rather than the template.

The last one exists because of measured failures. With an answer template beside the cue, 28 of 30 answers collapsed onto a single move, while the probability mass stayed at 0.99, so only the spread of answers revealed it. A bare line reading “Round N” above the cue dropped the mass from 0.99 to 0.61.

2.2 Completeness

A separate check compares rows on disk against what each arm's own configuration says it should produce, rather than against numbers written into the checker. It also catches duplicate cells across files, which is what a double submit looks like, and two measurement regimes written into one directory. All nine arms reported clean. Two initially showed duplicates from an accidental second submit and were cleaned before anything was read.

2.3 Derivability, on the relation arms

On the arms built around the counter opponent, a round is only answerable once the history has actually demonstrated all three beats facts. A fact is demonstrated by any decisive round. This is stored per row and the clean view drops anything below three.

regime	counter	tft	mirror	cycle	random
generating modes	93.0	79.1	81.4	81.4	72.1
logprob modes	93.3	94.3	94.6	90.2	91.2

Usable rows, percent. Identical to one decimal across every verb, which no stochastic filter would produce. This is a design guarantee, not attrition.

Why the two regimes differ. They probe different rounds. The generating modes probe every round to 30 and then every fifth, so about 67 percent of their probes fall in rounds 1 to 30. The logprob modes probe every round, putting 29 percent there. Early rounds are exactly where the three facts have not yet been shown, so the generating modes oversample the unanswerable region.

3. Run 1: what went wrong

Run 1's generating arms were contaminated and this is the single most important methodological result in the project.

opponent	untruncated	truncated	gap
cycle	91.9	29.3	-62.6
tft	60.1	29.0	-31.1
mirror	68.5	44.1	-24.4
random	34.7	32.9	-1.8

Accuracy split by whether the response was cut off.

How to read this. If truncation were simply losing rows at random, the two columns would be similar. They are not. Truncated rows score at chance. And random is the control that proves the mechanism: it had the most truncation of any opponent, at 48 percent, and the smallest gap, because on random there is no signal to lose in the first place.

The reason was that the retry fallbacks were never fired when the reasoning ran out of budget, the parsing exception handling was built on wrong conditions.

4. Run 2: accuracy by verb

4.1 The base checkpoint, no hint

verb	level	cycle	mirror	tft	random
forecast	high	74.9	53.5	48.9	32.4
infer	high	75.2	56.3	52.0	32.2
predict	high	71.5	52.9	49.2	31.6
anticipate	mid	76.5	59.3	52.1	34.9
estimate	mid	73.5	53.1	51.3	31.6
expect	mid	83.8	61.1	52.8	34.5
guess	low	74.7	49.4	50.5	35.2
imagine	low	87.1	61.3	54.6	33.9
suppose	low	87.3	61.2	52.9	35.8
say	neutral	80.5	56.6	60.3	34.0
none	none	86.9	59.9	56.9	34.0

Accuracy, percent. Random has a hard ceiling of 33.3.

4.2 Free generation, no hint and then with the pointer

verb	cycle L0	mirror L0	tft L0	cycle L2	mirror L2	tft L2
forecast	100.0	43.4	41.9	44.2	89.1	69.8
infer	95.3	46.5	38.8	48.8	85.3	69.8
predict	97.7	46.5	36.4	50.4	88.4	62.0
anticipate	95.3	37.2	32.6	40.3	85.3	72.1
estimate	93.0	36.4	34.1	44.2	85.3	65.9
expect	95.3	39.5	38.8	41.1	82.9	76.0
guess	98.4	32.6	41.1	48.1	86.0	69.8
imagine	87.6	29.5	43.4	44.2	89.1	66.7
suppose	86.0	28.7	45.7	45.0	84.5	69.0
say	97.7	34.9	39.5	39.5	90.7	74.4
none	97.7	41.9	44.2	55.8	88.4	79.1

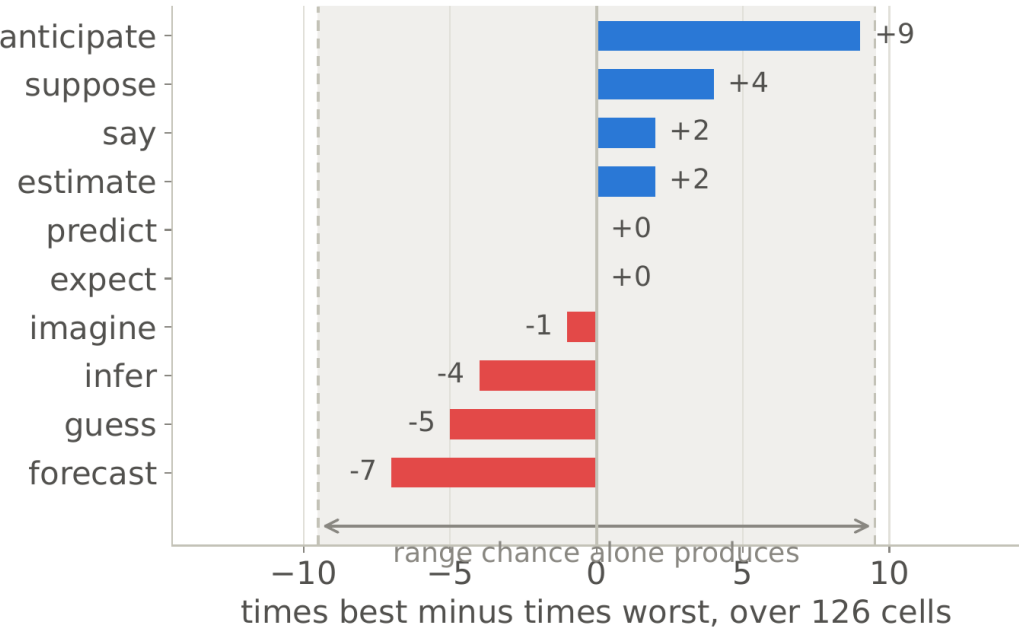
L0 is no hint. L2 adds the pointer, which is false for cycle and true for mirror and tft.

This is the single most informative table in the document. Read it left to right within an opponent. Cycle is essentially solved without the hint, at 86 to 100 percent, and collapses to 39 to 56 percent once a false hint is added. Mirror runs the other way, from the high twenties and thirties to the high eighties. Tft the same, smaller.

Read it top to bottom instead and there is no verb story at all. On cycle L0 the spread across verbs is 14 points and the top is forecast at 100; on mirror L0 the top is infer and predict tied at 46.5; on tft L0 the top is suppose at 45.7, which is a low verb. Nothing holds across the three columns.

4.3 The ordering test

The per verb tables above are one cell each. The ordering test does the same comparison in every cell of every arm and counts how often each verb comes first and last.



How often each verb came first minus how often it came last, over 126 cells. The grey band is what chance alone produces.

verb	first	last	net
anticipate	21	12	+9
estimate	17	15	+2
imagine	16	17	-1
say	16	14	+2
expect	11	11	0
guess	10	15	-5
infer	10	14	-4
suppose	10	6	+4
forecast	9	16	-7
predict	6	6	0

126 cells. Under chance each verb is first 12.6 times.

How to read it. A verb with a real advantage would come first far more often than last. None does. Predict is exactly balanced. Infer, which the reasoning analysis had singled out, comes last more often than first. Simulating 126 draws over ten bins puts the expected maximum at 18.3 with a standard deviation of 1.97, so anticipate at 21 sits 1.4 standard deviations above the expected maximum, which is not significant.

29 of the 126 cells exceed their own noise floor and 8 exceed it by half again, but they contradict each other. In the teacher forced arm infer is the worst verb on cycle with no hint and the best on mirror with the pointer, a 17.5 point spread and a 22.9 point spread pointing opposite ways.

5. Run 2: the phrasing

form	the line, with the verb predict
question	What do you {verb} the other player will play in this round?
statement	{verb} the other player's move in this round.
statement_n	{verb} the other player's move in round 4.
taskverb	Your task is to {verb} the other player's move in this round.
anchor	Which move will the other player play in this round?

arm	mean change vs question	largest single change
base	-3.5	-15.2
teacher forced	+5.1	+25.7
free generation	about 0	+8.3

Each phrasing paired against the question form with the verb held, on the same round of the same transcript.

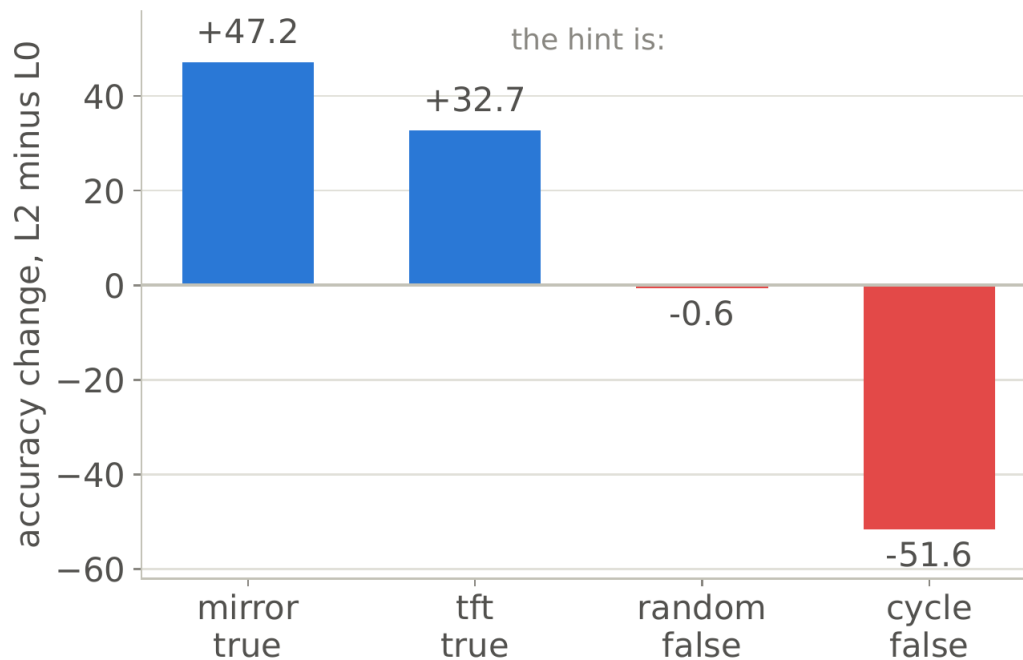
5.1 What the phrasing does move

form	L0	L2	L3
question	0.780	0.896	0.830
statement	0.870	0.890	0.826
statement_n	0.837	0.860	0.804
taskverb	0.614	0.655	0.694

Probability mass on legal move labels, base checkpoint.

taskverb has the lowest coverage of the four forms at every level of both logprob arms, without exception, sitting 12 to 23 points below the mean of the others. That is stable and reproducible..

6. Run 2: the pointer



Accuracy change from adding the pointer, in the arm where the model writes out its answer, question form.

opponent	hint is	no hint	with hint	change
mirror	true	39.5	86.7	+47.2
tft	true	36.2	68.9	+32.7
random	false	37.0	36.4	-0.6
cycle	false	96.2	44.6	-51.6

One sentence, added or not added.

Why the random row is the important one. The hint is false for both random and cycle. On cycle it costs 51.6 points. On random it costs 0.6. If the effect were simply that adding a sentence disturbs the model, random would move too. It does not, which means what is being measured is the hint being used rather than text being added.

Why the cycle row is the finding. Cycle is solved at 96.2 percent without the hint. The model has it. One false sentence takes it to 44.6, in the face of a history in front of it that contradicts the sentence on every line. It does not check the instruction against the evidence..

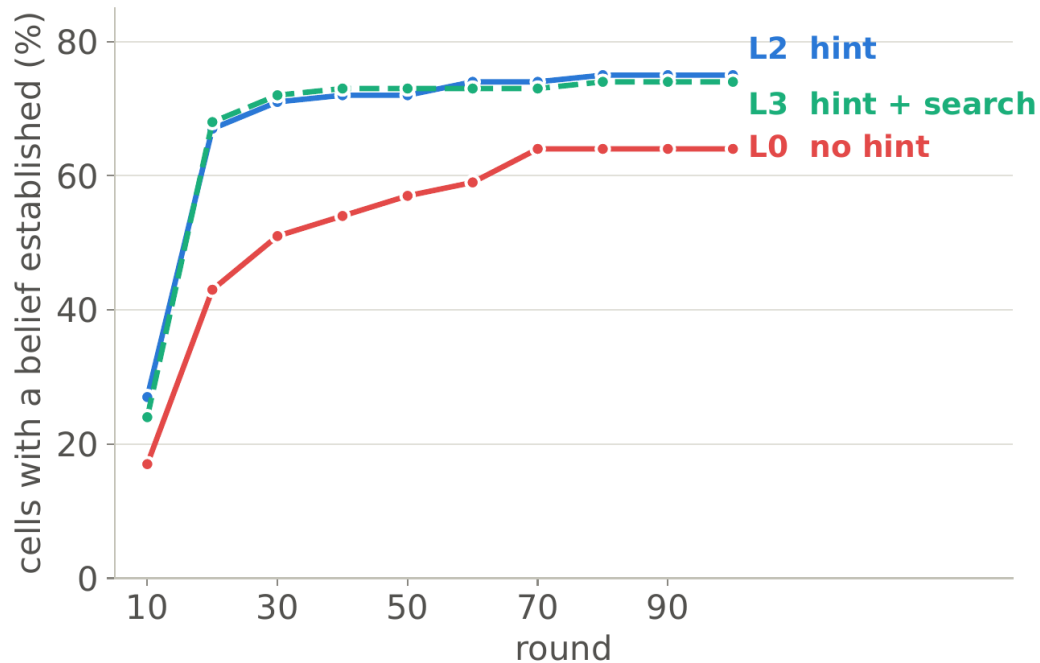
6.1 Splitting the hint in two

arm	cycle	mirror	tft	random
generating, pointer alone	-51.6	+47.2	+32.7	-0.6
generating, search added	+3.6	+4.2	-2.3	+4.6

Pointer alone is L2 minus L0. Search added is L3 minus L2.

What this says. The pointer does the work. Adding “find the pattern” on top of it buys between -2.3 and +4.6 in the generating arm.

7. Run 2: when the belief arrives



A belief is counted once the model has been rule correct on five consecutive probes.

opponent	reached	median round
counter	91.7%	5.0
tft	90.0%	11.0
cycle	88.3%	16.0
mirror	76.7%	11.0
random	8.3%	22.0

Random at 8.3 percent is the falsifier working. Five consecutive correct guesses at one in three happens by chance about 0.4 percent of the time per starting position, so a few cells hitting one across many positions is about right for noise.

8. The four reasoning databases

Four databases were singles out to look at the models raw reasoning and compare it per verb

database	opponent	harness	rows
IN mirror	mirror	free generation	3,741
RC mirror	mirror	rule stated and carried forward	2,580
IN cycle	cycle	free generation	3,741
IN tft	tft	free generation	3,741

8.1 The one place the hypothesis looks true

database	high	mid	low
IN mirror	45.5	37.7	30.2
IN cycle	97.7	94.6	90.7
RC mirror	50.8	41.9	49.6
IN tft	39.0	35.1	43.4

With no hint. Nine epistemic verbs grouped into three levels.

On mirror with no hint the ordering is clean, holds in all three phrasings and every stretch of the game, and survives controlling for response length, for truncation, and for dropping the short answers.

And here is why it does not carry. On cycle the gap is entirely two low verbs skipping the reasoning altogether, at 86.0 and 87.6 against 93 to 100 for everything else. On the second mirror harness the middle verbs come lowest, which is not an ordering. On tft there is nothing. The verb rankings from the two mirror harnesses correlate at about zero, and no pair of opponents agrees. And in run 2, across 126 cells, no ordering replicates at all.

8.2 What does replicate: the register

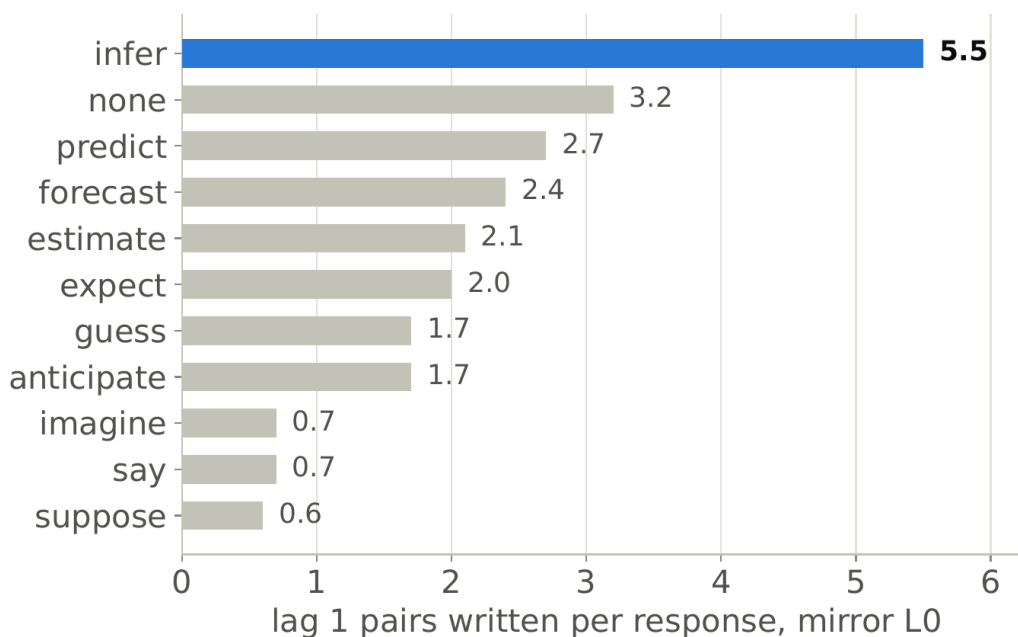
marker	mirror vs cycle	mirror vs tft	deduction
hedging	+0.77	+0.92	high verbs lowest
probability words	+0.82	+0.78	high verbs lowest
cycle words	+0.87	+0.65	high verbs highest
statistics words	+0.63	+0.53	high verbs lowest
row by row checks	0.00	+0.81	high verbs highest on mirror and tft
index notation	+0.50	+0.85	high verbs highest on mirror and tft

Rank correlation of the per verb profile across opponents. Hedging also replicates across the two mirror harnesses at +0.87.

High confidence verbs reason in the language of rules and low confidence verbs in the language of likelihood, and that pattern is repeated on every opponent.

Once a pointer is given, no profile is replicated showing that the pointer is what dictates the content and the verb stops shaping it.

8.3 Reading both columns



Lines where the model puts one of its own moves next to the opponent's at a one round offset, mirror with no hint.

verb	reads your column	tests a lagged relation	lag 1 pairs written
infer	82%	60%	5.5
predict	78%	54%	2.7

forecast	75%	52%	2.4
estimate	74%	44%	2.1
guess	72%	54%	1.7
expect	67%	43%	2.0
anticipate	64%	46%	1.7
say	62%	41%	0.7
suppose	40%	32%	0.6
imagine	40%	30%	0.7

mirror, no hint. The verb decides whether the model looks at your column at all.

- Infer leads and writes about twice as many lagged comparisons as the next verb, with the difference across verbs significant at $p = 0.0003$. Suppose and imagine barely look at all, which is consistent with these two being the verbs most likely to skip the reasoning and answer bare. In statement form, 74 to 86 percent of suppose's answers carry no reasoning at all. The rate depends on the form as well as the verb, and it is much lower elsewhere, 23 to 30 percent for imagine as an imperative, so the two effects are not the same thing.
- And doing it decides the answer. On mirror with no hint, responses that finish and make three or more lagged comparisons are right 84.7 percent of the time, against 32.9 percent for those that do not.
- But the lead is small in accuracy, and here is the funnel that eats it. Of infer's responses on mirror, 59.7 percent make a lagged comparison, 25.6 percent name the copy rule, and only 10.1 percent commit to it at the end. Those that commit are right 100 percent of the time. Three things leak: most lagged comparisons test the wrong mapping, many that name the right rule then verify it on same round pairs instead of previous round pairs and drop it, and of those that name the right rule without finishing on it, 13 of 20 were cut off.

8.4 Where the search starts

database	periodicity first, high / mid / low	your column first
IN mirror	75 / 64 / 50	0 to 1.6%
IN tft	79 / 61 / 50	0 to 1.6%
IN cycle	96 / 92 / 74	0 to 1.6%

First hypothesis family mentioned, with no hint.

The crux of the dilemma is that with nothing pointing there, your own column is essentially never the first thing the model considers, on any opponent, and with any verb. It is not that the model weighs the cross stream hypothesis and rejects it. It mostly never raises it. High verbs open with a cycle search; whereas mid and low verbs open with frequency counts about twice as often.

9. isolate2: one word swapped

The reasoning work above suggested infer was tied to the step that solves mirror. That is a hypothesis from four single seed databases, so it was tested directly, and the cross stream check is seen when compared to predict but it doesn't translate to a jump in accuracy.

9.1 The two prompts

ARM A

L1 Your task is to predict the opponent's next move from the history.

LN ... The opponent is not repeating a fixed sequence.

LS ... The opponent's move depends on something other than its own past moves.

L2 ... The opponent's move depends on what you played.

L3 ... The opponent's move depends on what you played in the previous round.

ARM B

The same five levels with "infer" in place of "predict".

9.2 Accuracy: almost nothing

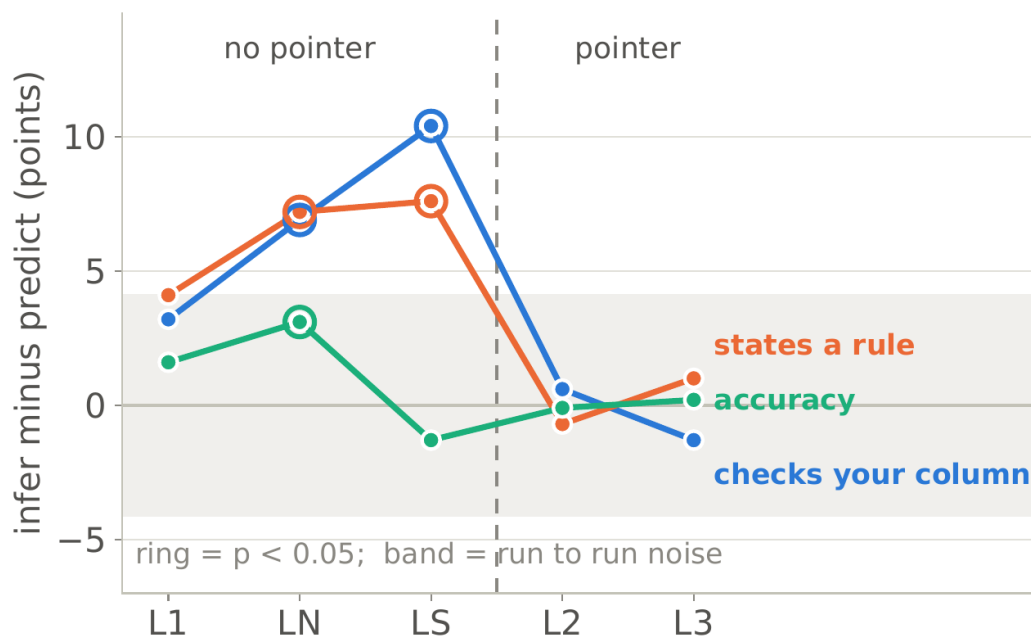
opponent	L0	L1	LN	LS	L2	L3
mirror	53.7 / 52.7	50.7 / 45.2	49.3 / 45.9	72.1 / 71.8	66.3 / 73.8	78.9 / 78.2
tft	34.7 / 34.0	35.0 / 35.7	42.2 / 34.0	50.3 / 54.1	38.4 / 39.8	52.0 / 55.4
cycle	97.3 / 97.6	98.3 / 93.9	95.9 / 95.9	92.9 / 96.3	77.6 / 69.4	52.7 / 50.7
selfbeat	96.6 / 96.6	96.3 / 97.3	95.9 / 93.2	94.6 / 94.9	72.4 / 72.4	51.4 / 50.7
fixed	100.0 / 99.7	100.0 / 100.0	98.6 / 97.6	100.0 / 99.3	100.0 / 99.7	95.2 / 94.2

infer / predict. Three of the 25 manipulated cells reach $p < 0.05$, against about 1.25 expected by chance.

Pooled over opponents, only LN differs, by 3.1 points at p 0.009. L1 is +1.6, LS is -1.3, L2 is -0.1 and L3 is +0.2, none of them significant. The control levels differ by 1.1 points on average and 4.1 at most, which is the honest measure of how much two runs of the same prompt differ.

On mirror specifically, the target opponent, L1 favours infer by 5.4 and LN by 3.4, neither significant.

9.3 Reasoning: the predicted shift



Infer minus predict, pooled over the five opponents. Rings mark significance. The band is the noise floor from the control levels.

measure	L1	LN	LS	L2	L3
checks your column	+3.2 (2)	+6.9 (5)	+10.4 (5)	+0.6 (3)	-1.3 (0)
states a rule	+4.1 (4)	+7.2 (5)	+7.6 (5)	-0.7 (1)	+1.0 (4)
mentions your moves	+0.4 (2)	+1.1 (4)	+2.0 (5)	+1.3 (5)	-0.9 (2)
response length, chars	+342 (5)	+311 (4)	+542 (4)	-190 (1)	-73 (3)
lag 1 scan	+1.7 (2)	+1.7 (4)	+3.2 (5)	+4.0 (4)	-1.8 (1)

Pooled over opponents. Every row is percentage points except response length, which is characters. The bracket is how many of the five opponents moved in that direction.

At LN and LS, where the prompt rules out the opponent's own column without naming yours, infer raises cross stream checking and rule statements on all five opponents. At L2 and L3, where the prompt names your column, everything goes to zero. The verb matters exactly in the gap the instruction leaves and nowhere else.

And the statistics back it up rather than resting on one cell. Across the manipulated levels, 78 of 225 paired tests come out significant. Across the control levels, where both arms see the same prompt, 6 of 180 do, which is 3.3 percent and is simply the false positive rate.

9.4 Why so little of it reaches the score

cell	checks your column	right when it checks	right when it does not	rows needing rescue
mirror L1	55.4 / 46.6	64.4 / 51.8	33.6 / 39.5	13.6 / 7.5
mirror LN	62.2 / 54.4	56.8 / 53.1	36.9 / 37.3	5.8 / 3.7
tft L1	55.4 / 40.5	37.4 / 36.1	32.1 / 35.4	12.9 / 4.8
tft LN	59.5 / 49.7	41.1 / 37.0	43.7 / 31.1	7.5 / 1.0

infer / predict.

On mirror, infer checks more often and its checks convert better, 64.4 against 51.8 at L1, but the net gain is only 5.4 points. Part of that is lost to more rows ending without an answer line and needing a rescue parse, 13.6 against 7.5. On tft the extra checking does not convert at L1 at all, because tft needs the extra beats step on top of reading your column. And neither arm ever starts from your column at the levels that do not name it.

One useful negative is that on the opponents where your column is the wrong place to look, infer takes the bait more often but pays nothing for it. Accuracy stays at 93 to 100 percent, which means the model checks your column, finds nothing, and goes back to the opponent's own column. That is the right behaviour.

9.5 Predict being used in Infers reasoning.

The model adopts the word “infer” in about only 37 percent of responses at L1 and LN, and rarely elsewhere, while the predict arm uses “predict” in 67 to 93 percent. The infer arm still writes “predict” in 55 to 77 percent of its own responses. So the verb changed the behaviour without being taken up or conversely when the model reasons with the wrong synonym even though the correct one is given the results (rise/fall- can't be exact as this specific detail just came to me and the cluster is down to directly test it).

10. Caveats

- The reasoning results are from only four databases, a fraction of the total results.
- The generating arms of run 2 used a single seed/ and per seed variation has been observed before.
- Only one model size of one model family is used, and the results could be different in different model/architectures.
- The teacher forced arm reads 100 percent below the floor coverage on every opponent and should not be compared directly against the others.